

UNIVERSIDAD DEL VALLE DE GUATEMALA

Facultad de Ingeniería - CC3067 Redes

Proyecto 1

Uso de un protocolo existente

Reporte técnico final

Servidor MCP local y remoto - Análisis con Wireshark

Estudiante

Ihan Marroquín - 23108

Catedrático

Kevin Velásquez

Guatemala, 20 de septiembre de 2026

Resumen ejecutivo

El proyecto implementa un chatbot de consola que se comunica con un LLM por medio de la API de Anthropic y utiliza herramientas MCP. Todo el ciclo MCP se programó manualmente sobre JSON-RPC 2.0, sin FastMCP ni SDKs que oculten el protocolo. El anfitrión conserva el contexto, registra las interacciones y solicita confirmación antes de ejecutar herramientas de escritura.

El servidor propio resuelve un caso de mantenimiento industrial: consulta máquinas, repuestos y órdenes de trabajo. La misma lógica se expone de dos formas: localmente por stdio y remotamente mediante Streamable HTTP. La versión remota incorpora sesiones, validación de Origin, token Bearer, encabezado de versión y empaquetado Docker para Google Cloud Run. Doce pruebas automatizadas verifican servicio, ciclo de vida, ambos transportes y el cliente HTTP.

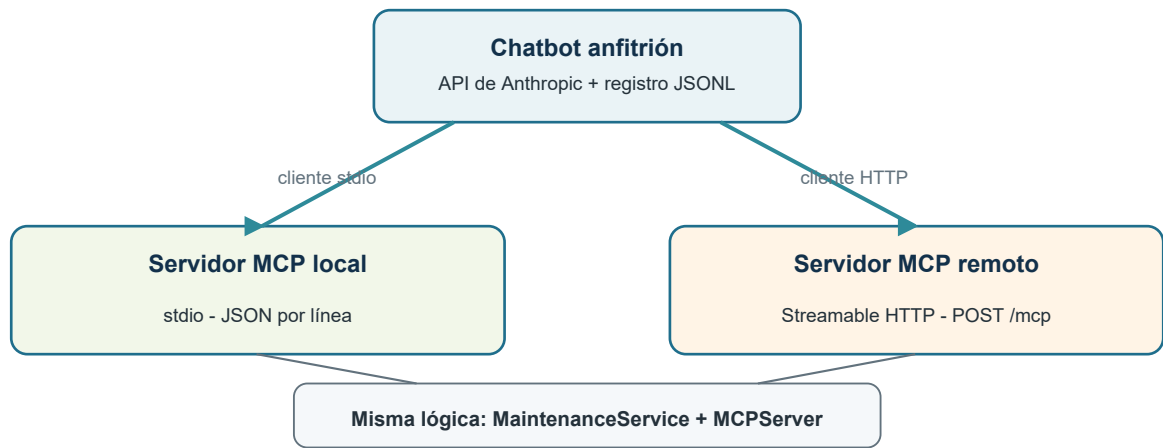


Figura 1. Arquitectura implementada y reutilización de la lógica del servidor.

Alcance entregado

- Chatbot de consola con contexto de sesión y herramientas de Anthropic.
- Clientes MCP manuales para stdio y Streamable HTTP.
- Servidor industrial de seis herramientas, disponible local y remotamente.
- Registro JSONL de solicitudes, notificaciones y respuestas.
- Contenedor, guía de Cloud Run y verificador remoto sin costo de API.
- Procedimiento reproducible y análisis por capas para Wireshark.

Índice

Sección	Contenido
1	Inciso 8 - Especificación del servidor MCP
2	Inciso 9 - Captura y análisis con Wireshark
3	Inciso 10 - Conclusiones y comentario
4	Referencias y verificación

1. Inciso 8 - Especificación del servidor MCP

1.1 Datos generales

Elemento	Implementación
Nombre	industrial-maintenance-server
Versión	2.0.0
Protocolo	MCP 2025-11-25 sobre JSON-RPC 2.0
Servidor local	stdio; un objeto JSON UTF-8 por línea
Servidor remoto	Streamable HTTP; endpoint único POST /mcp
Salud	GET /health
Persistencia	Archivo JSON con escritura atómica y bloqueo entre hilos
Dependencias MCP	Ninguna; biblioteca estándar de Python

1.2 Caso de uso industrial

El servidor centraliza información de una planta de bebidas. Permite que un técnico consulte el estado de maquinaria, revise existencias y administre órdenes de mantenimiento desde lenguaje natural. La lógica de negocio permanece en el servidor; el LLM únicamente elige una herramienta y prepara argumentos estructurados.

1.3 Ciclo de vida y puntos de interacción

Método / endpoint	Función	Resultado
initialize	Negocia versión y capacidades	InitializeResult y datos del servidor
notifications/initialized	Completa la sincronización	Sin respuesta JSON-RPC
ping	Comprueba disponibilidad	Objeto result vacío
tools/list	Descubre herramientas y esquemas	Lista de seis herramientas
tools/call	Ejecuta una herramienta validada	content, structuredContent, isError
POST /mcp	Transporta un mensaje remoto	HTTP 200 o 202
DELETE /mcp	Termina una sesión remota	HTTP 204

Antes de recibir notifications/initialized, el servidor rechaza operaciones normales con el error -32002. En HTTP, la inicialización crea un identificador seguro en MCP-Session-Id; cada petición posterior debe incluirlo junto con MCP-Protocol-Version: 2025-11-25.

1.4 Herramientas y parámetros

Herramienta	Parámetros	Uso y efecto
list_machines	status opcional	Lista máquinas; solo lectura
get_machine_status	machine_id requerido	Devuelve el registro de una máquina
list_work_orders	machine_id y status opcionales	Consulta órdenes; solo lectura
check_spare_part	query requerido	Busca repuestos y recomienda reabastecer
create_work_order	machine_id, issue, priority	Crea una orden; requiere confirmación
close_work_order	work_order_id, resolution	Cierra una orden; requiere confirmación

1.5 Validación y seguridad

- Los esquemas rechazan parámetros adicionales, catálogos inválidos, identificadores desconocidos y cadenas demasiado cortas.
- El anfitrión presenta servidor, herramienta y argumentos antes de una operación de escritura.
- La versión HTTP limita el cuerpo a 1 MiB, exige application/json y valida Accept.
- Origin se compara contra una lista explícita para reducir riesgo de DNS rebinding.
- MCP_AUTH_TOKEN protege el endpoint mediante Authorization: Bearer.
- El token y la clave de Anthropic se leen del entorno y nunca se incluyen en Git.

1.6 Ejemplo de llamada

```
{"jsonrpc": "2.0", "id": 3, "method": "tools/call", "params": {"name": "get_machine_status", "arguments": {"machine_id": "SELL-01"}}}
```

La respuesta conserva id: 3 y devuelve el estado de SELL-01 dentro de result.structuredContent. El mismo mensaje funciona por stdio o HTTP porque el manejador MCP es compartido.

1.7 Despliegue y verificación

El Dockerfile ejecuta el servidor como un usuario sin privilegios, escucha el puerto indicado por Cloud Run y guarda la demostración en almacenamiento efímero. La guía `deploy/cloud-run.md` documenta compilación, token, URL y límites de persistencia. `scripts.verify_remote` realiza initialize, tools/list y tools/call sin usar la API del LLM.

Comandos mínimos

```
python -m unittest discover -s tests -v
./run_http.ps1
python -m scripts.verify_remote
python -m src.chatbot --config config/servers_remote.json
```

Resultado de revisión: 12 pruebas automatizadas finalizadas en OK. Las pruebas cubren servicio, validación, ciclo MCP, stdio, HTTP, sesiones, seguridad de Origin, cliente remoto y registro de mensajes.

2. Inciso 9 - Captura y análisis con Wireshark

2.1 Metodología

La evidencia de contenido se obtiene ejecutando el servidor HTTP en 127.0.0.1:8000, capturando el adaptador loopback y usando el filtro de captura tcp port 8000. Luego se ejecuta scripts.verify_remote. Este escenario conserva exactamente los mensajes y el transporte de la versión en nube, pero permite inspeccionar JSON-RPC sin publicar credenciales ni descifrar TLS.

Una segunda captura contra Cloud Run documenta DNS, TCP y TLS en el puerto 443. El cuerpo MCP permanece cifrado, lo cual es el comportamiento esperado. Las instrucciones y filtros completos se encuentran en docs/WIRESHARK_ANALYSIS.md.

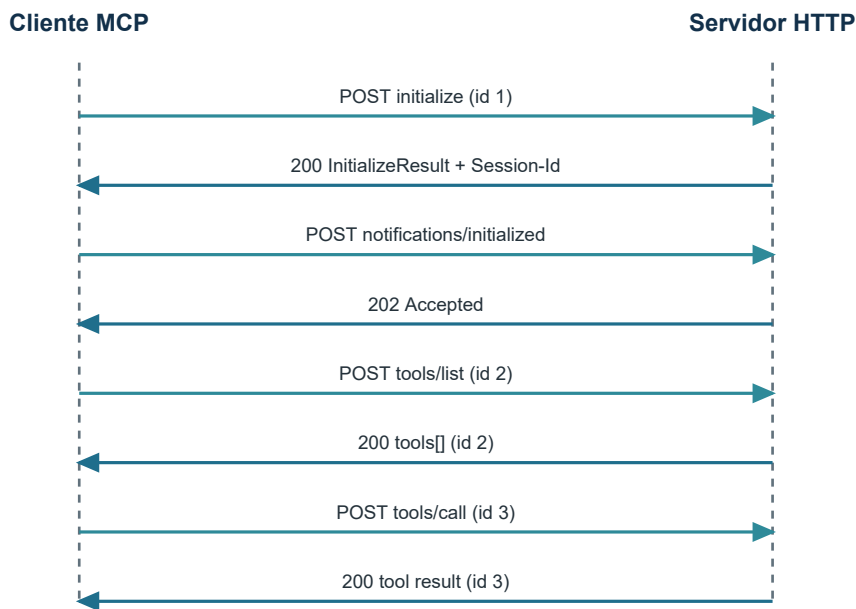


Figura 2. Secuencia determinista producida por scripts.verify_remote.

2.2 Clasificación de mensajes

Mensaje	Categoría	Identificación
initialize (id 1)	Sincronización y solicitud	Primera operación; negocia versión/capacidades
InitializeResult (id 1)	Respuesta	Mismo id y nuevo MCP-Session-Id
notifications/initialized	Sincronización y notificación	No tiene id ni respuesta JSON-RPC
tools/list (id 2)	Solicitud/petición	Solicita catálogo de herramientas
tools[] (id 2)	Respuesta	Seis definiciones con inputSchema
tools/call (id 3)	Solicitud/petición	Invoca list_machines
tool result (id 3)	Respuesta	Devuelve una máquina en warning

El identificador JSON-RPC correlaciona peticiones y respuestas. Una notificación no incluye id; el HTTP 202 solo confirma aceptación por el transporte y no constituye una respuesta JSON-RPC.

2.3 Análisis por capas

Capa	Qué sucede	Campos a evidenciar
Enlace	Loopback no cruza Ethernet física. En nube, la MAC destino es el siguiente salto local, no Cloud Run.	Tipo de enlace, longitud, MAC origen/destino y EtherType cuando aplique.
Red	IP identifica extremos lógicos y permite el enrutamiento. NAT puede traducir la dirección local en la prueba remota.	IPv4/IPv6, origen, destino, TTL/Hop Limit y protocolo TCP.
Transporte	TCP establece una conexión confiable, segmenta HTTP y confirma bytes con Seq/Ack.	Puerto efímero, 8000 o 443, SYN/SYN-ACK/ACK, ventana y retransmisiones.
Aplicación	HTTP transporta JSON-RPC. MCP define initialize, tools/list y tools/call. HTTPS cifra el caso remoto.	Método, ruta /mcp, códigos 200/202/204, encabezados MCP y cuerpo JSON.

Capa de enlace

La captura local permanece dentro del host. Dependiendo del controlador de captura, Wireshark presenta un encabezado loopback en lugar de Ethernet II. En la captura remota sí existe un enlace físico o inalámbrico; sus MAC pertenecen al equipo y al gateway. Los routers reemplazan estas direcciones en cada salto.

Capa de red

Las direcciones loopback identifican cliente y servidor local. En Cloud Run, DNS resuelve el dominio y los paquetes se encaminan hacia una IP pública. El campo de protocolo identifica TCP y TTL/Hop Limit evita ciclos indefinidos.

Capa de transporte

TCP comienza con SYN, SYN-ACK y ACK. El puerto del cliente es efímero; el destino es 8000 localmente o 443 en nube. Los números de secuencia, acuse y ventana proporcionan entrega ordenada y control de flujo. El filtro tcp.analysis.flags permite revisar retransmisiones o segmentos fuera de orden.

Capa de aplicación

Cada mensaje cliente-servidor viaja en un POST independiente. Accept anuncia JSON y SSE; este servidor responde JSON porque no necesita notificaciones iniciadas por el servidor. La sesión se conserva mediante MCP-Session-Id y se elimina con DELETE. JSON-RPC aporta estructura y correlación; MCP aporta el ciclo y la semántica.

2.4 Resultado del análisis

La secuencia confirma que MCP no es un protocolo de transporte independiente: utiliza JSON-RPC como formato de mensajes y, en este caso, HTTP sobre TCP/IP. También muestra que la sincronización debe completarse antes de descubrir o invocar herramientas. La captura remota agrega TLS entre HTTP y TCP para proteger credenciales y contenido.

3. Inciso 10 - Conclusiones y comentario

3.1 Conclusiones

- MCP separa el razonamiento del LLM de la ejecución real. El modelo selecciona una herramienta; el servidor valida y ejecuta la acción.
- JSON-RPC simplifica la correlación por id y distingue claramente solicitudes, respuestas y notificaciones.
- La lógica del servidor puede reutilizarse entre stdio y Streamable HTTP; el cambio principal se concentra en ciclo de conexión, sesiones y seguridad.
- El transporte remoto requiere controles ausentes en un subproceso local: Origin, autenticación, tamaño máximo, versión y terminación de sesión.
- Wireshark evidencia la separación de capas: enlace e IP entregan paquetes, TCP aporta confiabilidad, HTTP transporta y MCP define la conversación.
- Las pruebas automáticas son indispensables para comprobar que ambos transportes producen los mismos resultados sin consumir créditos del LLM.

3.2 Dificultades

La dificultad principal fue conservar el ciclo MCP sobre dos transportes. stdio mantiene un proceso y una secuencia natural de mensajes; HTTP crea una solicitud nueva por cada mensaje y necesita un identificador de sesión. También fue necesario diferenciar una notificación aceptada con HTTP 202 de una respuesta JSON-RPC y evitar que diagnósticos contaminen el canal de protocolo.

En el análisis de red, el reto conceptual fue no confundir direcciones MAC con extremos remotos y reconocer que HTTPS impide leer el cuerpo MCP. Por esta razón se definió una captura local controlada para el contenido y otra remota para DNS, TCP y TLS.

3.3 Lecciones aprendidas

- Diseñar primero la lógica de protocolo permitió compartirla entre servidores.
- Registrar mensajes en JSONL facilita comparar el comportamiento con una captura.
- Los esquemas de entrada y la confirmación humana reducen errores en operaciones persistentes.
- Un despliegue de demostración puede usar almacenamiento efímero, pero un sistema industrial real necesita base de datos, autenticación administrada, auditoría y alta disponibilidad.

3.4 Comentario personal

El proyecto permitió comprender que conectar un LLM con una herramienta no consiste solo en llamar una función. Existe un protocolo con negociación, capacidades, esquemas, correlación, seguridad y manejo de errores. Implementarlo manualmente hizo visibles decisiones que un SDK normalmente oculta y permitió relacionar programación de aplicaciones con conceptos concretos de redes.

4. Referencias y verificación

Recurso	Ubicación
MCP - Transportes 2025-11-25	modelcontextprotocol.io/specification/2025-11-25/basic/transport
MCP - Ciclo de vida	modelcontextprotocol.io/specification/2025-11-25/basic/lifecycle
MCP - Herramientas	modelcontextprotocol.io/specification/2025-11-25/server/tools
Wireshark User's Guide	wireshark.org/docs/wsug_html_chunked/
Código y pruebas	Repositorio Proyecto1_redes

Estado verificado

Comprobación	Resultado
Pruebas unitarias e integración	12/12 OK
Servidor stdio	Inicializa, lista y ejecuta herramientas
Servidor Streamable HTTP	Inicializa, crea sesión, lista, ejecuta y cierra
Verificador remoto	3 pasos PASS sin API del LLM
Docker	Configuración incluida; requiere daemon local o Cloud Build
Captura Wireshark	Procedimiento y matriz de evidencia incluidos